Prometheus — firmware 1.14.0

Documentação revisada em **19/10/2026**. Consulte também COMANDOS.md para `protetor:*`, `botao_status` e `audio_status`.

Navegação e protetor — 1.14.0

O botão agora é GPIO27/GND; a série gpio_digital_input usa pin="27". Consultas /metrics e do painel não acordam as carinhas nem reiniciam o tempo de inatividade. O protetor de robô não desliga o OLED, mantendo web e coleta ativas. As métricas de CPU, SD e demais grupos são mantidas.

Temperatura da CPU no Grafana

A métrica `esp_cpu_temperature_celsius` publica a temperatura interna do chip, em °C, a cada coleta. No painel do Grafana com a fonte Prometheus, use:

```
esp_cpu_temperature_celsius{job="esp32_p4"}
```

Escolha a unidade **Celsius (°C)** e a visualização Time series, Stat ou Gauge. Se houver várias placas, filtre também pelo label `instance`. Ajuste o job se tiver usado outro nome no servidor. Após gravar o firmware, aguarde a próxima coleta. Não é necessário alterar a porta ou cadastrar um novo alvo no Prometheus.

O comando `temp` e o exporter compartilham o mesmo driver do sensor. Falha de instalação/leitura resulta em `NaN` e em `esp_metric_available{group="cpu_temperature"}=0`, sem publicar um zero falso. Esta leitura é da CPU, separada da temperatura ambiente. Compilação e teste físico desta mudança ficam a cargo do usuário.

Padrão de conexão do M5NanoC6

O Prometheus consulta http://IP_DA_PLACA:PORTA/metrics. Não há envio ativo ou Pushgateway. A interface web e a API permanecem na porta **80**. As métricas anteriores, inclusive SD, permanecem iguais nesta revisão.

1. Abra **Prometheus** no menu da placa, em /prometheus na porta 80.
2. Informe o hostname/IPv4 do servidor, sem protocolo, caminho ou porta.
3. Escolha a **porta de métricas da placa** (padrão 80) e o **intervalo de coleta** de 5–3600 segundos (padrão 15).
4. Salve nas preferências. A porta é aplicada cerca de um segundo depois, sem reiniciar, redirecionar ou mudar a porta da interface.
5. Copie o YAML mostrado, ou baixe /prometheus.yml, e aplique no servidor.

O endereço do servidor é uma referência persistida, não um destino de envio, uma restrição de origem ou o endereço usado em targets. O alvo é o IP da placa. O intervalo é gravado no YAML e só altera a frequência real após aplicar/recarregar essa configuração no Prometheus. Salvar na placa não altera o servidor automaticamente.

Na porta 80, as métricas compartilham a interface. Com outra porta configurada, somente essa porta atende /metrics; a porta 80 retorna 404 para essa rota. O segundo servidor atende apenas /metrics, sem expor o painel/configuração. A porta TCP 3232 é reservada ao OTA deste projeto; as demais de 1–65535 são aceitas. A porta 4210 pode atender métricas HTTP junto dos comandos UDP, pois são protocolos diferentes. Use IP fixo ou reserva DHCP; a coleta funciona por Wi-Fi, Ethernet e no portal AP.

Persistência e migração

Um único valor config no namespace Preferences prometheus guarda a versão do formato, porta, intervalo e endereço. A escrita é conferida antes de usar os valores. As configurações da versão 1.13.0 preservam endereço e porta; o intervalo inicia em 15 s. O antigo campo de porta do servidor e a chave de ativação deixam de ser usados: assim como no M5NanoC6, a coleta fica sempre ativa. O botão de limpar Wi-Fi não apaga as configurações do Prometheus. Apagar toda a flash/NVS elimina configurações e contadores.

API

Todas as rotas de configuração continuam na porta 80:

- GET /api/prometheus/config: JSON com host, port (métricas), active_port, web_port (80), interval, mode (pull), scrape_config e persistent_counters_ok.
- POST /api/prometheus/config: formulário host, port e interval; exige o header X-ESP32-Command: 1. Retorna o mesmo JSON. Porta ativa pode mostrar o valor anterior durante o segundo de espera. Validação inválida: 400; NVS indisponível: 503; atualização/reinício em andamento: 409. Não há CORS habilitado.
- POST /api/prometheus: alias para o mesmo formato novo.
- GET /prometheus.yml: download da configuração de coleta.
- GET /metrics: texto Prometheus na porta ativa, sem cache. Quando a porta ativa não for 80, /metrics na web responde 404, sem redirecionamento.

Exemplo de coleta

```
scrape_configs:
  - job_name: 'esp32_p4'
    scrape_interval: 15s
    metrics_path: /metrics
    static_configs:
      - targets: ['192.168.1.50:80']
```

Substitua IP e porta pelos da placa. Se já existir scrape_configs, acrescente somente o item job_name e use um nome único. Recarregue o servidor após mudar porta ou intervalo. O exporter é HTTP sem autenticação, destinado à rede local; não contém senhas, conteúdo de arquivos ou SSIDs.

O formato continua Prometheus texto 0.0.4, UTF-8, com Content-Type e quebra de linha final, sem timestamps. Referências: formato oficial e configuração de coleta.

Identificação e disponibilidade

Métrica	Valor / interpretação
`esp_info`	1, labels `model` e `sdk`.
`esp_firmware_version`	1, label `version`.
`esp_ip_address`	1, labels `interface` (`wifi`/`ethernet`) e `address`. Sem IP: `0.0.0.0`.
`esp_mac_address`	1, labels `interface` e `address`.
`esp_status_online`	1 quando Wi-Fi STA ou Ethernet têm conexão/IP; 0 no AP de configuração isolado.
`esp_uptime_seconds`	Tempo ligado por relógio de 64 bits, sem o rollover de `millis()`.
`esp_last_restart_reason`	Código numérico de `esp_reset_reason()` do SDK.
`esp_boot_count`	Inicializações, persistidas em NVS uma vez por boot.
`esp_reboot_count`	Inicializações cujo motivo não foi `POWERON`; inclui brownout, reset, deep sleep etc.
`esp_watchdog_resets`	Inicializações por WDT de tarefa, interrupção ou geral; persistidas.
`esp_prefs_counters_persisted`	1 se os três contadores foram gravados; 0 indica falha.

Código de reset do SDK: 0 desconhecido, 1 power-on, 2 externo, 3 software, 4 panic, 5 WDT interrupção, 6 WDT tarefa, 7 WDT geral, 8 deep sleep, 9 brownout, 10 SDIO. Motivos adicionais dependem do SDK. Os contadores de boot/reboot/WDT viram `NaN` se NVS não puder ser aberta; uma falha de escrita é indicada por `esp_prefs_counters_persisted=0`. Apagar NVS reinicia os contadores.

Uma placa desligada não consegue publicar `esp_status_online=0`. Para detectar indisponibilidade, use a série `up{job="esp32_p4"}` criada pelo próprio Prometheus.

Wi-Fi, processador e memória

Métrica	Unidade / interpretação
`esp_wifi_rssi_dbm`	RSSI em dBm; `NaN` sem STA conectada.
`esp_wifi_channel`	Canal; `NaN` sem STA conectada.
`esp_wifi_disconnect_count`	Eventos STA desconectada desde o boot, incluindo desconexões solicitadas.
`esp_wifi_tx_power`	Potência configurada em dBm, convertida de quartos de dBm do SDK.
`esp_cpu_frequency_mhz`	Frequência em MHz.
`esp_cpu_usage_percent`	Média dos dois cores na última janela de pelo menos 1 s, por tempo não idle do FreeRTOS; `NaN` antes da segunda amostra ou em SDK incompatível.
`esp_cpu_temperature_celsius`	Temperatura interna da CPU em °C; `NaN` se o sensor não puder ser lido.

Métrica	Unidade / interpretação
`esp_task_count`	Número atual de tarefas FreeRTOS.
`esp_free_rtos_tasks`	Alias de `esp_task_count`, não quantidade de vagas de tarefas.
`esp_heap_free_bytes`	Heap interno livre.
`esp_heap_total_bytes`	Heap interno total.
`esp_heap_minimum_free_bytes`	Menor heap livre desde o boot.
`esp_psram_free_bytes`	PSRAM livre; 0 se não habilitada.
`esp_psram_total_bytes`	PSRAM total; 0 se não habilitada.
`esp_flash_usage_percent`	Tamanho do aplicativo dividido pela capacidade física da flash, em %. Não mede arquivos nem espaço livre da partição OTA.

Energia, sensores, relés, GPIO e MQTT

O projeto atual não tem drivers/configuração para monitor elétrico, bateria, sensores ambientais, relés, entradas analógicas ou MQTT. As séries abaixo são expostas como **NaN**, sem simular leituras. `esp_metric_available{group="..."}` vale 0 para `energy`, `environment`, `relays`, `analog` e `mqtt`.

Grupo	Métricas sem fonte instalada
Energia	`esp_voltage_v`, `esp_current_ma`, `esp_power_w`, `esp_battery_percent`, `esp_battery_voltage_v`
Ambiente	`sensor_temperature_celsius`, `sensor_humidity_percent`, `sensor_pressure_hpa`, `sensor_air_quality`
Relés	`relay_state`, `relay_activation_count`, `relay_runtime_seconds`
Analógico	`gpio_analog_value`
MQTT	`mqtt_messages_sent`, `mqtt_messages_received`, `mqtt_publish_errors`

`mqtt_connected=0` porque não há cliente MQTT. Não representa falha em um broker configurado. A temperatura interna do processador não é apresentada como ambiente. O exporter somente lê GPIOs já configurados pela aplicação; não altera seus modos nem aciona saídas durante uma coleta. A tarefa de áudio usa os pinos indicados no README.md, sem acrescentar uma métrica de áudio nesta versão.

As séries reais `gpio_digital_input{pin="27"}` e `gpio_digital_output{pin="1"}` leem, respectivamente, o botão externo e o LED já configurados pelo projeto. A entrada usa pull-up: **0 = botão pressionado**. Os pinos seguem `DeviceConfig.h`, não ficam fixos no exporter. As disponibilidades de `wifi`, `cpu_usage`, `cpu_temperature` e `sdcard` também são publicadas.

SD e aplicação

O SD é amostrado no ciclo principal a cada **30 segundos**, fora de OTA. A consulta `/metrics` usa esse cache, sem percorrer diretórios nem escrever logs. Não há remontagem automática do cartão. Após remoção/falha, a

disponibilidade pode levar até o próximo teste de leitura para mudar. Valores de capacidade viram `NaN` quando a leitura da raiz/sistema de arquivos falha.

Métrica	Significado
`sdcard_mounted`	Última verificação de leitura/montagem, 0 ou 1.
`sdcard_capacity_bytes`	Capacidade física do cartão.
`sdcard_total_bytes`	Capacidade do sistema de arquivos.
`sdcard_used_bytes`	Uso do sistema de arquivos.
`sdcard_free_bytes`	Total menos usado.
`sdcard_usage_percent`	Uso / total × 100.
`sdcard_log_size_bytes`	Tamanho de `/log.txt`; 0 se ausente, `NaN` se não pôde ser aberto.
`sdcard_log_exists`	Presença do arquivo na última amostra.
`sdcard_log_writes_total`	Gravações de log com quantidade de bytes completa.
`sdcard_log_errors_total`	Tentativas de log sem SD, falhas de abertura ou escrita incompleta.
`sdcard_crash_reports_total`	Relatórios de crash gravados e verificados.
`sdcard_crash_errors_total`	Tentativas de arquivamento de crash que falharam.
`sdcard_last_sample_uptime_seconds`	Uptime da última amostra; permite observar sua idade.
`application_requests_total`	Requisições HTTP despachadas na web e exporter, inclusive 404 e a própria coleta.
`application_errors_total`	Dessas requisições, respostas HTTP >= 400. Não contabiliza erros semânticos de comandos respondidos com HTTP 200 nem conexões que não chegaram a ser despachadas.
`application_response_time_ms`	Duração do último handler HTTP concluído, incluindo envio da resposta; não é média nem percentil e não inclui recepção prévia do corpo/upload.

O snapshot mostra os contadores anteriores à requisição `/metrics` atual. Contadores de rede, aplicação, log e crash reiniciam a cada boot. Os de log/crash cobrem operações de `StorageManager`, não todos os comandos manuais de arquivos. Os três contadores de boot são a exceção persistida em NVS.

Validação

Testes de interface: `node --test tests/*.test.cjs`. Compilação: Arduino ESP32 3.3.12, alvo `esp32:esp32:esp32p4`, partições OTA padrão. A compilação desta revisão será feita pelo usuário, conforme solicitado. Os 34 testes de interface passaram na validação mais recente. Testes reais de coleta com Prometheus, persistência após falta de energia, mudança de porta, remoção do SD e leituras físicas exigem a placa.